

JUnit Testing Exercises

COMPLETE IMPLEMENTATION BLUEPRINT & REFERENCE GUIDE

Overview of Included Solutions:

- **Exercise 1:** Environment configuration using Maven `pom.xml` infrastructure.
- **Exercise 3:** Detailed assertion variations to validate application state transformations.
- **Exercise 4:** Lifecycle fixtures (`@Before` / `@After`) and the Arrange-Act-Assert architectural pattern.

Exercise 1: Setting Up JUnit

To establish a reliable unit testing environment, you must introduce the framework dependencies into your project pipeline. Below is the complete configuration required for a standardized Apache Maven project layout.

1. Maven Dependency Configuration (pom.xml)

Add the following block inside your `<dependencies>` element to pull down the JUnit 4 compiler artifact and testing harness:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

2. Standard Directory Setup

Ensure that production modules and test environments are explicitly compartmentalized following standard build conventions:

```
my-java-project/
├── pom.xml
└── src/
    ├── main/
    │   └── java/      <-- Business applications live here
    └── test/
        └── java/     <-- JUnit test cases live here
```

Exercise 3: Assertions in JUnit

Assertions provide descriptive failure reporting mechanisms to ensure execution branches match targeted domain logical specifications. Below is the fully structural execution class containing basic assertions.

```
import org.junit.Test;
import static org.junit.Assert.*;

public class AssertionsTest {

    @Test
    public void testAssertions() {
        // Assert equals: valid equality check (Expected, Actual)
        assertEquals(5, 2 + 3);

        // Assert true: validates condition resolves to a true flag
        assertTrue(5 > 3);

        // Assert false: validates condition resolves to a false flag
        assertFalse(5 < 3);

        // Assert null: ensures reference target pointer is unassigned
        assertNull(null);

        // Assert not null: confirms an instantiated object reference exists
        assertNotNull(new Object());
    }
}
```

Exercise 4: AAA Pattern & Lifecycle Test Fixtures

To minimize duplicate variable instantiations and maintain execution state isolation, tests utilize standard lifecycle fixtures alongside structural patterns.

The Arrange-Act-Assert (AAA) Architectural Anatomy:

1. **Arrange:** Set up target components, inject stubs, and establish initialization state.
2. **Act:** Execute the specific operation or method under testing observation.
3. **Assert:** Check state variations against expected outcome invariants.

Implementation Blueprint

This class implements an isolation model where every individual test gets a fresh data collection environment via `@Before` and resets resources via `@After`.

```

import org.junit.Before;
import org.junit.After;
import org.junit.Test;
import static org.junit.Assert.*;
import java.util.ArrayList;

public class ListLifecycleTest {

    // Test Fixture: Target reference shared contextually across operations
    private ArrayList<String> sampleList;

    @Before
    public void setUp() {
        // Executes before EACH separate test run to establish clear context
        sampleList = new ArrayList<>();
        System.out.println("Lifecycle Status: New array list infrastructure instantiated.");
    }

    @After
    public void tearDown() {
        // Executes directly following EACH test completion for garbage control
        sampleList.clear();
        System.out.println("Lifecycle Status: Fixture teardown executed cleanly.");
    }

    @Test
    public void testListElementAddition() {
        // 1. ARRANGE
        String operationalToken = "JUnit";

        // 2. ACT
        sampleList.add(operationalToken);

        // 3. ASSERT
        assertEquals(1, sampleList.size());
        assertTrue(sampleList.contains("JUnit"));
    }
}

```